

Initial Proposal

Here is your high-altitude weather balloon project documentation reformatted into a clean, highly scannable, and professional engineering proposal.

Mission To The Edge Of Space: Avionics & Flight Computer Proposal

Mission Statement

To design, fabricate, and fly a custom embedded flight computer to **~100,000 ft (30 km)** via a high-altitude weather balloon. The payload must operate autonomously for ~2 hours in near-vacuum, -60°C conditions, transmit live telemetry via LoRa, capture Earth's curvature, and safely recover via a GPS-tracked parachute.

The Ultimate Deliverable: Custom-designed avionics that flew to the edge of space and came back.

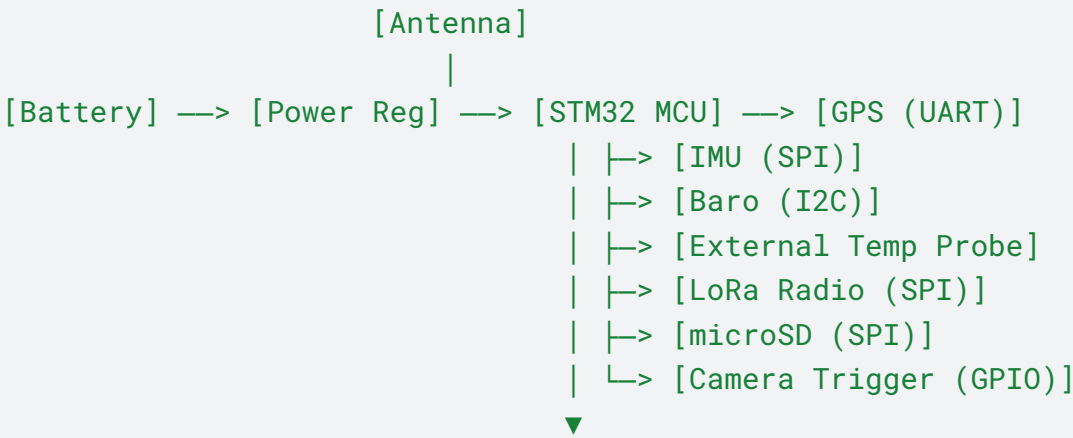
System Architecture

The payload consists of **three independent subsystems** housed within a single chassis:

- Flight Computer (Custom PCB):** STM32-based core that controls the mission state machine and telemetry.
- Imaging Module:** Raspberry Pi Zero 2W + Camera, triggered externally by the Flight Computer.
- Recovery System:** Parachute assembly integrated with independent GPS trackers.

System Block Diagram

None



[Status LEDs]

Note: The Pi Zero camera runs its own OS independently; the Flight Computer simply toggles a GPIO pin every N seconds to capture a photo.

Hardware Design (EE Work)

Specifications: Custom 4-layer PCB, ~75×50 mm.

Layer Stackup: Signal — Ground — Power — Signal (*Solid ground plane required for clean RF return paths*).

Component Selection

Component	Part Number	Engineering Justification
MCU	STM32F411CEU6	Cortex-M4 with FPU, abundant peripherals, excellent documentation.
GPS	u-blox MAX-M10S	Critical: Supports >18 km altitude. Standard GPS locks out at ~60k ft due to COCOM export limits.
Radio	RFM95W LoRa (915 MHz)	Long-range, low-power, operates on license-free US ISM band.
IMU	ICM-42688-P	Ultra-low noise gyro/accelerometer for reliable attitude logging during float.
Barometer	BMP390	Validated for high-altitude profiles and wide temperature swings.

Ext. Temp	TMP117	External digital probe to accurately capture ambient air temperature.
Storage	SanDisk Industrial 32GB	Industrial grade; rated for wide operating temperatures (-40°C to 85°C).
Power	4× Samsung INR18650-35E	2S2P configuration. Lithium-ion chemistry holds voltage at -30°C far better than LiPo.
Regulator	TPS63070 Buck-Boost	Dynamic 3.3V regulation as cell voltages drop rapidly in extreme cold.
ESD Protection	TVS Diodes	Placed on GPS, RF, and USB lines to survive static buildup on antennas.



Key Design Decisions to Defend in Interviews

- **$\lambda/4$ Monopole LoRa Antenna:** Measures exactly 8.2 cm at 915 MHz, paired with a dedicated PCB ground plane.
- **Buck-Boost Topology:** Chosen over standard buck regulators because cold-induced battery voltage sag will drop below nominal requirements.
- **Industrial SD Cards:** Standard consumer microSD cards heavily corrupt when exposed to temperatures below 0°C.



Firmware Architecture (Embedded Work)

Software Modules

1. **HAL / Drivers:** Low-level configuration for GPIO, UART, SPI, I2C, ADC, Timers, and Watchdog.
2. **Sensor Layer:** Drivers and data parsers for the IMU, Barometer, Temperature sensor, and GPS NMEA strings.
3. **Storage:** FATFS integration featuring periodic data flushes and write-after-write verification.
4. **Radio:** LoRa packet transmission featuring sequence numbering, CRC checks, and FEC utilizing the [RadioLib](#) library.

5. **Telemetry Packetizer:** Packs all vital sensor vectors into a fixed-size, 48-byte transmission payload.
6. **Camera Trigger:** Hardware GPIO toggle mapping to a strict 30-second interval during ascent.
7. **Power Management:** Deep-sleep routine entries during low-dynamic float states.
8. **Watchdog (IWDG):** Hardware watchdog to auto-recover and reboot the system from sensor brown-outs.

Mission State Machine

None

```
[ PAD ] —> Boot, device initialization, and cold GPS lock.
|
[ ARMED ] —> Triggered via physical button or magnetic switch.
|
[ ASCENT ] —> Positive altitude delta detected. Telemetry TX @ 1
Hz.
|
[ FLOAT ] —> Apogee reached. Telemetry drops to 0.2 Hz to preserve
power.
|
[ BURST ] —> Sudden descent trigger detected (>50 m/s drop).
|
[ DESCENT ] —> Target falling. Telemetry TX updates to 0.5 Hz.
|
[ LANDED ] —> Altitude stable for >60 seconds. Enters ultra-low
power beacon mode.
|
[ BEACON ] —> Emits GPS coordinates every 60 seconds indefinitely
for recovery.
```

Firmware Elevator Pitch: *"A real-time state machine driving an embedded avionics platform through a 2-hour mission in extreme environmental conditions, featuring autonomous failure-mode handling and a recoverable post-landing beacon."*

Mechanical, Packaging & Logistics

Payload Enclosure & Rigging

- **Chassis:** 1" thick insulated foam box lined with reflective Mylar (Interior: ~6"×4"×3").

- **Internal Layout:** Flight computer secured on central standoffs. Battery pack mounted flat at the base to maintain a low center of gravity. Pi Zero lens aligned with a small acrylic optical window.
- **External Elements:** LoRa antenna oriented vertically on top; GPS patch antenna pointing zenith; temperature probe extended out the lateral wall.
- **Recovery Hardware:** 30" nylon parachute, Nomex shock cords, and low-friction swivels attached directly to the balloon neck.
- **Weight Profile: < 4.0 lbs total mass.** This satisfies FAA Part 101 exemptions, requiring no formal FAA certifications.

Core Logistics Procurement

- **Launch Site:** Central California Valley farmland. Safe distance (≥ 5 miles) from airports and major highways; predictable wind patterns.
- **Gas Lift:** ~150 cu ft cylinder of Helium (Party City or local welding supply). Lift calculated via Archimedes' principle: $\text{Payload Weight} \times 1.5 \times \text{positive lift}$.
- **Balloon & Parachute:** Kaymont 1200g latex balloon paired with a 30" nylon recovery parachute from HABstore.

July 17

12-Week Project Timeline

None

💰 [W1-W3: Design & Fab] → 🛠️ [W4-W6: Bring-up & Test] → 🚀 [W7-W10: Integration & Test Launch] → 📡 [W11-W12: Main Mission]

Phase 1: Design & Fabrication

- **Week 1:** Assign roles (PCB, Firmware, Ops). Start KiCad schematics. Order STM32 Nucleo-F411 dev board. Read FAA Part 101 rules.
- **Week 2:** Complete schematics and clear Electrical Rules Check (ERC). Cross-verify Bill of Materials (BOM). Begin layout. Initialize basic UART/LED blinking on the Nucleo board.
- **Week 3:** Complete 4-layer PCB layout. Export Gerbers and clear Design Rules Check (DRC). **Submit to JLCPCB by Wednesday.** Firmware lead completes IMU and Barometer test drivers.

Phase 2: Bring-Up & Local Testing

- **Week 4:** Commercial parts arrive. Complete GPS NMEA parser and baseline LoRa TX code. Conduct a 5-mile ground-range line-of-sight radio test.
- **Week 5:** PCB arrives. Hand-assemble via hot-plate reflow. Perform initial power-on and peripheral verification. *Note: Expecting 1 bodge wire / firmware workaround.*
- **Week 6: Go/No-Go Board Respin Decision.** If unrecoverable, order a revised board Monday (8-day turnaround). If clear, port firmware from Nucleo to the custom PCB. Begin environmental freezer tests (-15°C).

Phase 3: Integration & Test Flights

- **Week 7:** Finalize state machine integration. Test Pi Zero camera triggers. Validate SD card write resiliency by abruptly pulling power during active logs.
- **Week 8:** Conduct an 24-hour continuous cold soak. Build the physical Mylar foam enclosure, perform a final mass check (< 4 lbs), and finalize launch permissions.
- **Week 9: Test Launch (Low Altitude).** Launch a scaled-down 300g balloon to ~25 km. This low-risk flight validates telemetry, tracking, and recovery workflows end-to-end.

Phase 4: Main Flight Operations & Closeout

- **Week 10:** Process test flight feedback. File formal FAA NOTAM (Notice to Air Missions). Track daily trajectory predictions via predict.sondehub.org.
- **Week 11:** 🚀 **MAIN LAUNCH WINDOW.** Deploy the 1200g balloon to 100k+ ft. Mobilize two chase vehicles tracking real-time LoRa telemetry to recover the downrange payload.
- **Week 12:** Process sensor logs, create altitude/temperature telemetry plots, compile curvature photos into a technical report, and open-source the project repository on GitHub.

⚠ Risk Mitigation Matrix

Identified Risk	Probability	Proactive Mitigation Strategy
PCB Respin Required	High (~70%)	Buffer built into Week 6. Firmware runs concurrently on generic Nucleo dev hardware to avoid losing time.
Total Loss of Payload	Moderate (~20%)	Conduct a low-altitude test flight first. Implement redundant tracking systems (LoRa + standalone Iridium satellite backup).
Cold-Induced System Crash	Moderate	Environmental torture testing via dry-ice/freezer cycles prior to final assembly.
GPS Altitude Lock Failure	Low	Explicitly selected the u-blox MAX-M10S to bypass standard civilian COCOM limits.

Battery Depletion	Low	Implemented cold-hardy Lithium-ion 18650 cylindrical cells encased in insulated thermal foam.
--------------------------	-----	---

Budget Breakdown

Budget Item Description	Minimum Flight Cost	Recommended "Moonshot" Flight Cost
Custom PCB Fabrication + Components (2 Flight Boards)	\$220	\$220
Kaymont 1200g Latex Balloon	\$100	\$100
Helium Gas Fill (~150 cu ft)	\$90	\$90
Parachute, Nomex Shock Cord, & Rigging Lines	\$40	\$40
Raspberry Pi Zero 2W + Camera Module	\$40	\$40
u-blox MAX-M10S GPS Module	\$35	\$35
LoRa RF Modules + Antennas (x2 for Flight + Base Station)	\$30	\$30

18650 Batteries + High-Retention Holder	\$30	\$30
Insulated Payload Structural Materials	\$25	\$25
STM32 Nucleo Development Board	\$25	\$25
Incidentals (Tape, Connectors, Helium Tank Deposit)	\$50	\$50
RockBLOCK 9603 Iridium SBD Satellite Tracker (<i>Stretch</i>)	—	\$260
Test Flight Consumables (300g Balloon + Helium) (<i>Stretch</i>)	—	\$90
TOTAL PROJECT INVESTMENT	~\$775	~\$1,035

📌 **Funding Ask Strategy:** A tight **\$500 bare-minimum budget** allows for a single high-risk flight with zero safety buffers. The optimal funding request is **\$800** to ensure high recovery confidence through an initial test launch and secondary satellite tracking.

Why This Secures Engineering Internships

- **Mixed-Signal PCB Design with RF:** Demonstrates an understanding of impedance matching, ground planes, and high-frequency routing—skills highly valued by hardware teams.
- **Ruggedized Embedded Firmware:** Proves you can write production-ready code that handles harsh environments, unexpected power drops, and state management.

- **The Ultimate Talking Point:** *"My custom hardware survived a -60°C near-vacuum environment at 30 km altitude."* It mimics NASA-aligned workflows and proves you can take a complex engineering project from schematic to flight.

[IMPORTANT] Initial Revisions ft. Gemini

This is already an exceptionally strong proposal. It reads like a capstone project or a professional aerospace prototype. However, if you are presenting this to a review board for funding, or using it to impress an interviewer, there are a few technical vulnerabilities, logistical realities, and firmware edge cases you should tweak.]

Here are the specific optimizations I would make across your design:

1. Hardware & Electrical Tweaks

- **The Battery Chemistry Choice:** You noted that Lithium-ion (18650) holds voltage better than LiPo. While true compared to *some* cheap LiPos, standard Li-ion chemistries still suffer massive capacity drops (up to 40–50%) and voltage sag at **-60°C**. For aerospace applications, **Lithium Iron Phosphate (\$LiFePO_4\$)** or **Lithium Manganese Dioxide (\$LiMnO_2\$ primary cells like Energizer Ultimate Lithiums)** are the gold standards because their internal resistance doesn't skyrocket as violently in extreme cold.
- **GPS Antenna Orientation:** You specified a "GPS patch antenna pointing zenith." Patch antennas require a massive, stable ground plane to function well and can lose lock if the payload spins or tilts violently (which weather balloons do during ascent and burst). Consider upgrading to a **helical GPS antenna**—they have an omnidirectional radiation pattern that handles payload sway much better.

2. Firmware & Edge Case Tweaks

- **The "Burst" State Logic:** Your state machine detects a burst via a sudden descent trigger ($>50 \text{ m/s}$ drop). Be careful here: right at the moment of burst, the air is so thin that the payload will virtually free-fall, easily exceeding 50 m/s . However, as it hits thicker air lower down, the parachute will slow it down significantly. Ensure your state machine doesn't get confused if the descent speed drops back *below* your threshold later in the flight.
- **LoRa Duty Cycle & Bandwidth:** You have telemetry firing at 1 Hz during ascent. Depending on your LoRa Spreading Factor (SF) and bandwidth settings (needed to achieve that 5+ mile line-of-sight range), a 48-byte payload might take several hundred milliseconds to transmit. A 1 Hz rate might violate local ISM band duty cycle regulations or saturate your airtime. I'd recommend throttling the baseline ascent telemetry to **0.25 Hz or 0.5 Hz (every 2 to 4 seconds)** to ensure the transmitter doesn't overheat and you stay legally compliant.

3. Logistical & Budget Realities

- **Helium Pricing:** You budgeted \$90 for 150 cu ft of Helium. Unless you have a specific hookup via a university lab, industrial Helium prices have spiked wildly over the last few years. 150 cu ft of pure lab/welding-grade Helium is more likely to run you **\$150 to \$250** today. (Party City gas is often a Helium-Air mix and won't give you the lift you calculated via Archimedes' principle).
- **The "Funding Ask" Math:** Your budget total says **Minimum: ~\$775**, but your Funding Ask paragraph says *"A tight \$500 bare-minimum budget..."* Make sure those numbers match perfectly so a sharp-eyed interviewer or professor doesn't catch the discrepancy. Change the text to reflect that your *absolute bare minimum* is ~\$775.

4. Re-shaping for the Interview (The Pitch)

- **Add a "Thermal Management Strategy":** In a near-vacuum, there is no air to conduct heat. Ironically, components like your buck-boost regulator and the Pi Zero can *overheat* if they run hard, even though the outside air is -60°C. Mentioning **"Vacuum Thermal Management via internal copper thermal mass dissipation"** as a design challenge you solved will absolutely blow a hardware interviewer away.

Would you like help re-writing the specific sections (like the state machine logic or the budget text) to seamlessly bake these upgrades into your original layout?

Here is the fully revised engineering proposal. The technical vulnerabilities, budget math, firmware edge cases, and thermal challenges have been seamlessly integrated to make this airtight for funding boards and interviewers.

[REDO 1] Mission to the Edge of Space: Avionics & Flight Computer Proposal

Mission Statement

To design, fabricate, and fly a custom embedded flight computer to **~100,000 ft (30 km)** via a high-altitude weather balloon. The payload must operate autonomously for **~2 hours** in near-vacuum, **-60°C** conditions, transmit live telemetry via LoRa, capture Earth's curvature, and safely recover via a GPS-tracked parachute.

The Ultimate Deliverable: Custom-designed avionics that flew to the edge of space and came back.

System Architecture

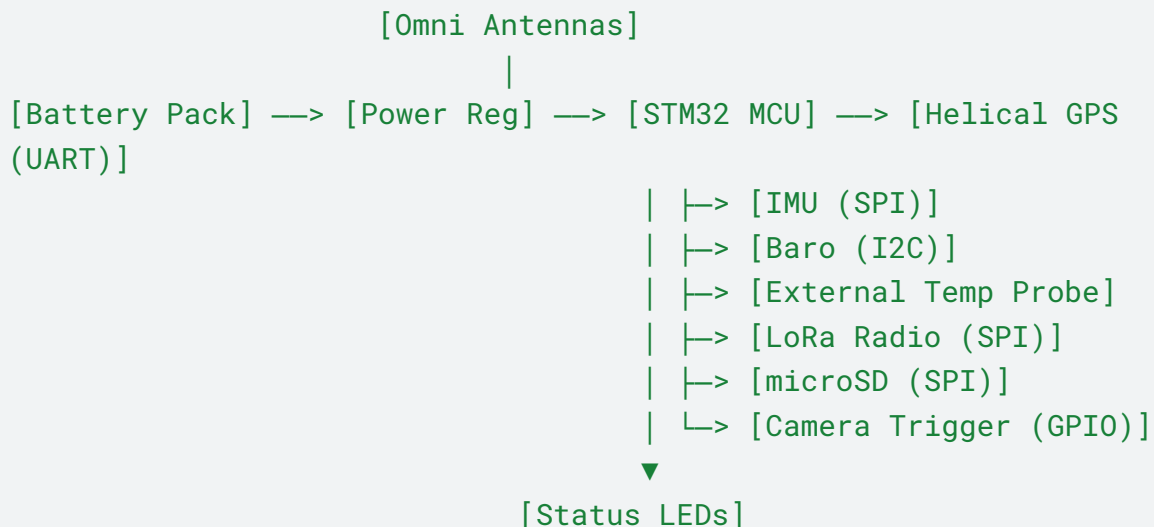
The payload consists of three independent subsystems housed within a single chassis:

1. **Flight Computer (Custom PCB):** An STM32-based core that controls the mission state machine and telemetry.
2. **Imaging Module:** A Raspberry Pi Zero 2W + Camera, triggered externally by the Flight Computer.
3. **Recovery System:** A parachute assembly integrated with independent GPS trackers.

System Block Diagram

Plaintext

None



Note: The Pi Zero camera runs its own OS independently; the Flight Computer simply toggles a GPIO pin every 30 seconds to capture a photo.

Hardware Design (EE Work)

- **Specifications:** Custom 4-layer PCB, ~75×50 mm.
- **Layer Stackup:** Signal — Ground — Power — Signal (*Solid ground plane required for clean RF return paths*).

Component Selection

Component	Part Number	Engineering Justification

MCU	STM32F411CEU6	Cortex-M4 with FPU, abundant peripherals, excellent documentation.
GPS	u-blox MAX-M10S	Critical: Supports >18 km altitude. Standard GPS locks out at ~60k ft due to COCOM export limits.
GPS Antenna	Helical Omni-Antenna	Replaced patch antenna to maintain satellite lock during violent payload spin/sway.
Radio	RFM95W LoRa (915 MHz)	Long-range, low-power, operates on license-free US ISM band.
IMU	ICM-42688-P	Ultra-low noise gyro/accelerometer for reliable attitude logging during float.
Barometer	BMP390	Validated for high-altitude profiles and wide temperature swings.
Ext. Temp	TMP117	External digital probe to accurately capture ambient air temperature.
Storage	SanDisk Industrial 32GB	Industrial grade; rated for wide operating temperatures (-40°C to 85°C).
Power	4× Energizer L91 (\$LiMnO ₂) no	Primary Lithium cells. Outperforms LiPo/Li-ion at -60°C with negligible internal resistance spikes.

Regulator	TPS63070 Buck-Boost	Dynamic 3.3V regulation to handle low-temperature battery voltage sag.
ESD Protection	TVS Diodes	Placed on GPS, RF, and USB lines to survive static buildup on antennas.



Key Design Decisions to Defend in Interviews

- **Vacuum Thermal Management:** In a near-vacuum, there is no air to conduct heat away from components. The PCB utilizes expanded copper planes and heavy thermal vias to sink heat away from the buck-boost regulator and the Pi Zero, turning the board itself into a thermal mass to prevent overheating.
- **$\lambda/4$ Monopole LoRa Antenna:** Measures exactly 8.2 cm at 915 MHz, paired with a dedicated PCB ground plane.
- **Primary Lithium Chemistry (LiMnO_2):** Chosen over standard Lithium-ion or LiPo cells because its chemistry maintains voltage stability and retains capacity down to -60°C without requiring heavy, power-hungry internal heaters.
- **Industrial SD Cards:** Standard consumer microSD cards heavily corrupt when exposed to temperatures below 0°C .



Firmware Architecture (Embedded Work)

Software Modules

- **HAL / Drivers:** Low-level configuration for GPIO, UART, SPI, I2C, ADC, Timers, and Watchdog.
- **Sensor Layer:** Drivers and data parsers for the IMU, Barometer, Temperature sensor, and GPS NMEA strings.
- **Storage:** FATFS integration featuring periodic data flushes and write-after-write verification.
- **Radio:** LoRa packet transmission featuring sequence numbering, CRC checks, and FEC utilizing the [RadioLib](#) library.
- **Telemetry Packetizer:** Packs all vital sensor vectors into a fixed-size, 48-byte transmission payload.
- **Camera Trigger:** Hardware GPIO toggle mapping to a strict 30-second interval during ascent.
- **Power Management:** Deep-sleep routine entries during low-dynamic float states.

- **Watchdog (IWDG):** Hardware watchdog to auto-recover and reboot the system from sensor brown-outs.

Mission State Machine

Plaintext

None

```
[ PAD ] —> Boot, device initialization, and cold GPS lock.
|
[ ARMED ] —> Triggered via physical button or magnetic switch.
|
[ ASCENT ] —> Positive altitude delta. Telemetry TX @ 0.5 Hz
(Optimized for RF airtime regulations).
|
[ FLOAT ] —> Apogee reached. Telemetry drops to 0.2 Hz to preserve
power.
|
[ BURST ] —> Sudden descent trigger detected (>50 m/s drop).
Latches state to prevent resetting in thick air.
|
[ DESCENT ] —> Target falling. Telemetry TX updates to 0.5 Hz.
|
[ LANDED ] —> Altitude stable for >60 seconds. Enters ultra-low
power beacon mode.
|
[ BEACON ] —> Emits GPS coordinates every 60 seconds indefinitely
for recovery.
```

Firmware Elevator Pitch: *"A real-time state machine driving an embedded avionics platform through a 2-hour mission in extreme environmental conditions, featuring autonomous failure-mode handling, RF duty-cycle compliance, and a recoverable post-landing beacon."*

Mechanical, Packaging & Logistics

Payload Enclosure & Rigging

- **Chassis:** 1" thick insulated foam box lined with reflective Mylar (Interior: ~6"x4"x3").
- **Internal Layout:** Flight computer secured on central standoffs. Battery pack mounted flat at the base to maintain a low center of gravity. Pi Zero lens aligned with a small acrylic optical window.

- **External Elements:** Omnidirectional LoRa antenna oriented vertically on top; helical GPS antenna pointing zenith; temperature probe extended out the lateral wall.
- **Recovery Hardware:** 30" nylon parachute, Nomex shock cords, and low-friction swivels attached directly to the balloon neck.
- **Weight Profile:** < 4.0 lbs total mass. This satisfies FAA Part 101 exemptions, requiring no formal FAA certifications.

Core Logistics & Procurement

- **Launch Site:** Central California Valley farmland. Safe distance (≥ 5 miles) from airports and major highways; predictable wind patterns.
- **Gas Lift:** ~150 cu ft cylinder of pure industrial-grade Helium (sourced via local welding supply). Lift calculated via Archimedes' principle:

$$Lift = \text{Payload Weight} \times 1.5 \times \text{positive lift}$$
- **Balloon & Parachute:** Kaymont 1200g latex balloon paired with a 30" nylon recovery parachute from HABstore.



12-Week Project Timeline

Plaintext

None

💰 [W1-W3: Design & Fab] → 🛠️ [W4-W6: Bring-up & Test] → 🚀 [W7-W10: Integration & Test Launch] → 📡 [W11-W12: Main Mission]

Phase 1: Design & Fabrication

- **Week 1:** Assign roles (PCB, Firmware, Ops). Start KiCad schematics. Order STM32 Nucleo-F411 dev board. Read FAA Part 101 rules.
- **Week 2:** Complete schematics and clear Electrical Rules Check (ERC). Cross-verify Bill of Materials (BOM). Begin layout. Initialize basic UART/LED blinking on the Nucleo board.
- **Week 3:** Complete 4-layer PCB layout with dedicated internal thermal copper planes. Export Gerbers and clear Design Rules Check (DRC). Submit to JLCPCB. Firmware lead completes IMU and Barometer test drivers.

Phase 2: Bring-Up & Local Testing

- **Week 4:** Commercial parts arrive. Complete GPS NMEA parser and baseline LoRa TX code. Conduct a 5-mile ground-range line-of-sight radio test.
- **Week 5:** PCB arrives. Hand-assemble via hot-plate reflow. Perform initial power-on and peripheral verification. *Note: Expecting 1 bodge wire / firmware workaround.*
- **Week 6:** Go/No-Go Board Respin Decision. If unrecoverable, order a revised board Monday (8-day turnaround). If clear, port firmware from Nucleo to the custom PCB. Begin environmental freezer tests (-15°C).

Phase 3: Integration & Test Flights

- **Week 7:** Finalize state machine integration. Test Pi Zero camera triggers. Validate SD card write resiliency by abruptly pulling power during active logs.
- **Week 8:** Conduct an 24-hour continuous cold soak. Build the physical Mylar foam enclosure, perform a final mass check (< 4 lbs), and finalize launch permissions.
- **Week 9: Test Launch (Low Altitude).** Launch a scaled-down 300g balloon to ~25 km. This low-risk flight validates telemetry, tracking, and recovery workflows end-to-end.

Phase 4: Main Flight Operations & Closeout

- **Week 10:** Process test flight feedback. File formal FAA NOTAM (Notice to Air Missions). Track daily trajectory predictions via predict.sondehub.org.
- **Week 11: 🚀 MAIN LAUNCH WINDOW.** Deploy the 1200g balloon to 100k+ ft. Mobilize two chase vehicles tracking real-time LoRa telemetry to recover the downrange payload.
- **Week 12:** Process sensor logs, create altitude/temperature telemetry plots, compile curvature photos into a technical report, and open-source the project repository on GitHub.

! Risk Mitigation Matrix

Identified Risk	Probability	Proactive Mitigation Strategy
PCB Respin Required	High (~70%)	Buffer built into Week 6. Firmware runs concurrently on generic Nucleo dev hardware to avoid losing time.
Total Loss of Payload	Moderate (~20%)	Conduct a low-altitude test flight first. Implement redundant tracking systems (LoRa + standalone Iridium satellite backup).
Vacuum Overheating	Moderate	Embedded deep copper thermal planes within the 4-layer stackup to distribute heat away from switching regulators without air conduction.

GPS Altitude Lock Failure	Low	Explicitly selected the u-blox MAX-M10S to bypass standard civilian COCOM limits.
Battery Voltage Sag	Low	Abandoned Li-ion/LiPo for primary Lithium \$LiMnO_2\$ cells that remain operational at extreme negatives.

Budget Breakdown

Budget Item Description	Minimum Flight Cost	Recommended "Moonshot" Cost
Custom PCB Fabrication + Components (2 Flight Boards)	\$220	\$220
Kaymont 1200g Latex Balloon	\$100	\$100
Welding-Grade Helium Gas Fill (~150 cu ft Cylinder)	\$180	\$180
Parachute, Nomex Shock Cord, & Rigging Lines	\$40	\$40
Raspberry Pi Zero 2W + Camera Module	\$40	\$40
u-blox MAX-M10S GPS Module	\$35	\$35

LoRa RF Modules + Antennas (x2 for Flight + Base Station)	\$30	\$30
Energizer Lithium Primary Cells (\$LiMnO ₂) + Holder	\$20	\$20
Insulated Payload Structural Materials	\$25	\$25
STM32 Nucleo Development Board	\$25	\$25
Incidentals (Tape, Connectors, Helium Tank Deposit)	\$60	\$60
<i>RockBLOCK 9603 Iridium SBD Satellite Tracker (Stretch)</i>	—	\$260
<i>Test Flight Consumables (300g Balloon + Helium) (Stretch)</i>	—	\$150
TOTAL PROJECT INVESTMENT	~\$775	~\$1,180

📌 **Funding Ask Strategy:** A tight \$775 budget allows for a baseline flight with standard tracking. The **optimal funding request is \$1,180**, which introduces critical aerospace-grade margins through a low-altitude test flight and secondary satellite tracking backup.



Why This Secures Engineering Internships

- **Mixed-Signal PCB Design with RF:** Demonstrates an understanding of impedance matching, ground planes, and high-frequency routing—skills highly valued by hardware teams.

- **Thermal and Environmental Systems Engineering:** Proves you can design for anomalous operating conditions like zero-convection thermal dissipation and sub-zero battery environments.
- **Ruggedized Embedded Firmware:** Proves you can write production-ready code that handles harsh environments, unexpected power drops, and state management.
- **The Ultimate Talking Point:** *"My custom hardware survived a -60°C near-vacuum environment at 30 km altitude."* It mimics NASA-aligned workflows and proves you can take a complex engineering project from schematic to flight.

Revisions

HAB Flight Computer — Final Bill of Materials v3

newAdded

changedModified

optionalCan cut if over budget

SSI coveredNot from your budget

5V main rail

TPS63070 (buck-boost from battery)

Powers Pi Zero directly + feeds lower rails

3.3V logic rail

AP2112K-3.3 (LDO from 5V)

STM32, IMU, LoRa, APRS, temp, SD

1.8V GPS rail

XC6206P182MR (LDO from 3.3V)

MAX-M10S GPS module only

PCB / Electronics\$157

Part	Part number	Notes	Qty	Unit	Total
MCU Cortex-M4, main brain	STM32F411CEU6	Bare IC, 3.3V rail	1	\$10	\$10
Crystal 8MHz External clock for STM32	ABM8-8.000MHZ	Cold-stable oscillator reference	1	\$1	\$1
GPS Module >18km altitude bypass	u-blox MAX-M10S	1.8V rail — cannot connect to 3.3V directly	1	\$35	\$35
GPS Patch Antenna Passive ceramic patch	Taoglas FXP73	Pairs with MAX-M10S via U.FL, faces up	1	\$4	\$4
GPS Level Shifter 3.3V ↔ 1.8V UART bridge	TXS0102 new	STM32 UART is 3.3V logic, GPS UART is 1.8V — need translation	1	\$0.80	\$0.80
LoRa Module (flight) 915 MHz primary downlink	RFM95W	3.3V rail, SPI to STM32	1	\$10	\$10
LoRa Module (ground) 915 MHz ground station RX	RFM95W	Connects to Nucleo dev board	1	\$10	\$10

APRS Radio Module VHF 144.390 MHz backup	Dorji DRA818V	3.3V rail, UART config + DAC audio in	1	\$8	\$8
VHF Low-Pass Filter Harmonic suppression	Mini-Circuits or discrete	Required by FCC for ham TX	1	\$3	\$3
IMU Sensor 6-axis gyro + accel	ICM-42688-P	Bare IC, SPI, 3.3V rail	1	\$6	\$6
Ext. Temp Probe Ambient digital sensor	TMP117	±0.1°C, I2C, 3.3V rail optional	1	\$4	\$4
Buck-Boost 5V Main power rail	TPS63070 change d	Now set to 5V output (resistor change only) — feeds 3.3V LDO + Pi Zero directly	1	\$4	\$4
LDO 3.3V Logic rail from 5V	AP2112K-3.3 chan ged	Was -5.0 variant. Now takes 5V in → 3.3V out. Input safely within 6V max rating.	1	\$0.5 0	\$0.50
Polyfuse 2A Resettable fuse	MF-MSMF200	Main power line, trips before battery vents	1	\$0.3 0	\$0.30
Reverse Polarity MOSFET P-ch at power input	DMG2305UX	Protects all ICs if battery reversed	1	\$0.3 0	\$0.30
Bulk Input Capacitor 100µF at battery input	Tantalum or electrolytic	Smooths LoRa TX spikes, prevents STM32 resets	2	\$0.5 0	\$1
Power Switch SPDT External on/off on box wall	Any panel mount	Power up on pad without opening box	1	\$1.5 0	\$1.50
Power LED + resistor 3.3V rail indicator	Generic	Visual confirmation board is live	1	\$0.2 0	\$0.20
microSD Holder Telemetry log only	Molex 104031	SPI to STM32, no photos stored here	1	\$1	\$1
SWD Debug Header 5-pin 1.27mm	Generic	Mandatory for firmware flashing	1	\$1	\$1

PCB Fabrication 4-layer, JLCPCB pool	Custom Design	5 copies per order, plan 1 respin	2	\$15	\$30
Passives & Protection Caps, resistors, TVS, ferrites	Various (LCSC)	Includes ~20 decoupling caps, audio divider resistors, crystal load caps	1	\$25	\$25

Embedded / Dev\$100

Part	Part number	Notes	Qty	Unit	Total
Dev Board Firmware proto + ground station RX	STM32 Nucleo-F411RE	Doubles as ground station during flight — connects to RFM95W #2	1	\$25	\$25
Imaging Computer Linux, camera stack, SSDV encode	Raspberry Pi Zero 2W	5V direct from main rail, GPIO trigger + UART back to STM32	4	\$15	\$15
Camera Module 42MP, autofocus, IMX708	Pi Camera Module 3	CSI-2 ribbon to Pi Zero, full res saved + downscaled for SSDV	4	\$25	\$25
Action Camera Cinematic post-recovery footage	Runcam 2 / Thumb Pro	External velcro mount, no electrical connection	1	\$45	\$45
Storage — Pi photos Pi Zero boot + image storage	SanDisk Industrial 32GB	Goes in Pi Zero built-in SD slot	4	\$15	\$15
Storage — STM32 log Telemetry only, no photos	SanDisk Industrial 8GB	Goes in PCB microSD holder via SPI	1	\$10	\$10
SSDV Library Image packetizer	github.com/fsphil/ssdv	Open source, runs on Pi Zero	—	\$0	\$0
Firmware Toolchain IDE + libraries	STM32CubeIDE / OSS	Free	—	\$0	\$0
Ham License Exam Required for APRS TX	ARRL / local club	One weekend study, valid 10 years	1	\$15	\$15

Mechanical / Ops SSI covered\$347

Part	Part number	Notes	Qty	Unit	Total
Primary Flight Balloon	Kaymont 1200g	Standard HAB, correct for 100k ft target	1	\$100	\$100
Recovery Parachute	HABstore / Apogee 30"	Right size for sub-4lb payload	1	\$30	\$30
Rigging Accessories	Various	Nomex cord, swivels, flight line	1	\$10	\$10
Helium Gas Fill	Local Welding Supply	~150 cu ft, get slightly extra	1	\$90	\$90
Flight Batteries		pre-protected 2S Li-ion pack	4		
Battery Chassis	2S2P 18650 Holder	High-retention cell holder	1	\$6	\$6
Enclosure Materials	Local hardware store	1" foam + Mylar lining	1	\$25	\$25
Antennas 915MHz	Amazon / Digikey	One on payload, one at ground station	2	\$5	\$10
VHF Antenna 144MHz	Any 144MHz whip	Payload only — APRS TX	1	\$5	\$5
Hand Warmers	Any grocery store	Inside payload box, battery insurance	4	\$0.50	\$2
Operational Misc.	Various	Tape, zip ties, NOTAM filing, gas deposits	1	\$50	\$50

Grand total

Electronics only (PCB + Embedded): **\$257** — within your \$200–300 budget

Power rails verified · all voltages within spec · 3 new parts added · 2 parts changed

\$604

\$347 SSI + \$257 your budget

STM32F411CEU6

- runs at 100 megahertz
- no linux (operating system)

8MHz Crystal ()

- the crystal is the stable temperature compensated clock reference
- without it, the stm32's internal oscillator would drift significantly with temperature -60 degrees celsius (drift by several percents)

u-blox MAX-M10S

- GPS receiver module
 - Talks to satellites and output position data over UART to STM32

GPS Patch Antenna (**Taoglas FXP73**)

- the MAX-M10S has a U.FL connector
 - this antenna plugs into it

RFM95W

- the LoRa radio transceiver module
- **SPI**
- RF signal transmitted at 915 MHz
- Second one for GND

Dorji DRA818V

- VHF FM transceiver module covering 134-174 MHz
 - **specifically 144.390 MHz for APRS**

ICM-42688-P

- this is IMU
- measures rotational rate, linear acceleration
- detects bursts when balloon pops at altitude (it'll show on accelerometer)

TMP117

- its a temperature sensor
- can measure +- 0.1 degrees C -40 degrees C to 150 degrees C

TPS63070

- its a buck boost
- For 5V rail

AP2112K-3.3 (add to BOM)

- Creates the 3.3V rail

MF-MSMF250/16X-2

- Protects against huge current spikes for battery

DMG2305UX

- Protects against us being retarded and plugging the battery in the wrong way

XC6206P182MR (LDO)

- Creates the 1.8V rail for MAX-M10S module

microSD Card Holder

- connects to STM32 over SPI
- log sensor data, gps coordinates, and flight events

***Two SD cards though — only one custom holder for STM32 for data, the other holder is built in Pi Zero

SWD Debug Header

- programming and debugging interface
- using this thing for programming on STM32 (uploads code)

Raspberry Pi Zero 2W

- has camera interface

- job is to receive a trigger signal from STM32 to tell when to capture photo using pi camera
- interprets the images
- sends data back to STM32

Pi Camera Module 3

- takes photos

TXS0102

- Allows us to communicate between GPS module (1.8V) and the STM (tx pin) (STM runs at 3.3V)

Runcam

- its own camera
- probably GoPro

Communication Protocols

SPI: IMU, LoRa module, SD card

I2C: temperature sensor

UART: GPS and HDRS

PCB Architecture

Two boards: flight computer and Pi-Zero

- Pi-Zero is premade - we're making only one board

GND Station

Laptop → USB cable → STM32 Nucleo → RFM95W module → 915MHz antenna

3 dif types antenna

915MHz – 2 – GND and Payload

144MHz – 1 – for APRS

Taoglas FXP73 GPS patch – 1 – for GPS

General Format:

One Custom PCB, Everything Hanging Off It

****NO INTERNAL CLOCK**

On the PCB itself (soldered directly):

- STM32F411 — the brain
 - Pin count? (32 vs 48 vs 64)
- ICM-42688-P — IMU
- TMP117 — temp sensor
- TPS63070 — 3.3V regulator
- AP2112K — 5V regulator
- microSD holder — telemetry log
- SWD header — firmware flashing
- Crystal 8MHz — clock reference
- All passives (caps, resistors, ferrites) [book?]

Modules that sit on or connect to the PCB:

- RFM95W → via U.FL coax → 915MHz whip antenna (exits box wall) transmitter
- DRA818V → via U.FL coax → 144MHz whip antenna (exits box wall) transmitter
- MAX-M10S → via U.FL coax → Taoglas FXP73 GPS patch (on top of box) receiver

External boards connected via wires/JST:

- Raspberry Pi Zero 2W → connected via 5V power + UART + GPIO trigger
 - Pi Camera Module 3 → plugs into Pi via CSI-2 ribbon cable, lens faces out through foam cutout

Completely standalone, no electrical connection:

- Runcam — just velcroed to the outside of the box

Every Component — What It Actually Does

STM32F411CEU6 This is the brain. A 32-bit ARM Cortex-M4 microcontroller running at up to 100MHz with a hardware floating point unit (FPU). It runs your firmware — the flight state machine, sensor polling loop, radio packet assembly, GPS parsing, everything. It has no operating system, no Linux, no scheduler you didn't write. It just executes your code instruction by instruction at 100 million cycles per second. The FPU matters because barometric altitude calculations and GPS coordinate math involve floating point numbers — without hardware FPU you'd do that in slow software emulation.

8MHz Crystal (ABM8) The STM32 has an internal oscillator but it drifts significantly with temperature — at -60°C it could be off by several percent. That sounds small but it means UART baud rates drift, SPI timing shifts, and your timers become inaccurate. The external crystal gives you a stable, temperature-compensated clock reference. The STM32 uses this as its phase-locked loop (PLL) input and multiplies it up to 100MHz internally. Two small load capacitors (typically 12–18pF, check crystal datasheet) sit on each crystal pin to ground — without these the crystal won't oscillate.

u-blox MAX-M10S A GNSS (GPS + GLONASS + Galileo) receiver module. Internally it has a dedicated RF front end, a SAW filter to reject out-of-band interference, a low-noise amplifier, and a baseband processor that tracks satellite signals and computes position. The key reason you chose this specific module is the COCOM limit bypass — most GPS chips hard-lock and output nothing above 18km altitude or above 515 m/s velocity, because of export control regulations designed to prevent use in ballistic missiles. The MAX-M10S is pre-certified for high-altitude balloon use and keeps outputting valid position data all the way to 80km. At 30km it's perfectly in range.

GPS Patch Antenna (Taoglas FXP73) A passive ceramic patch antenna that receives 1575.42 MHz L1 GPS signals. The MAX-M10S module has a U.FL connector — this antenna plugs into it. It needs to face the sky (upward) with a clear view of the hemisphere. Ceramic patch antennas are directional — they receive well from the upper hemisphere and poorly from below, which is exactly what you want since the satellites are above you.

RFM95W A LoRa radio transceiver module based on the Semtech SX1276 chip. It takes a data packet from the STM32 via SPI and converts it to a chirp spread spectrum (CSS) RF signal transmitted at 915MHz. CSS modulation is the magic — it spreads the signal across a wide bandwidth in a chirp pattern that can be decoded even when the signal is 20dB below the noise floor. This is why LoRa works at 100km+ with just 100mW of transmit power. The module includes the SX1276 chip, matching network, crystal, and U.FL connector — all the RF complexity is handled inside.

Dorji DRA818V A VHF FM transceiver module covering 134–174 MHz, used here specifically on 144.390 MHz for APRS. Internally it's a complete FM radio on a chip — it takes audio-frequency FSK data from the STM32, modulates

it onto the carrier, and transmits. For APRS your STM32 generates 1200 baud AFSK (Audio Frequency Shift Keying) tones — 1200Hz and 2200Hz audio — and feeds them into the DRA818V's audio input pin. The module does the FM modulation and power amplification. Output power is about 1W, significantly more than the RFM95W's 100mW, but it uses a conventional modulation scheme that has much lower receiver sensitivity than LoRa.

ICM-42688-P A 6-axis inertial measurement unit — 3-axis gyroscope plus 3-axis accelerometer on a single tiny die. The gyro measures rotational rate (degrees per second) around each axis. The accelerometer measures linear acceleration (g-forces) along each axis. On a balloon this tells you spin rate, pendulum swing, and orientation. More critically it detects burst — when the balloon pops at altitude, the payload goes from gentle upward acceleration to sudden freefall, which shows up instantly on the accelerometer. This can trigger your flight state machine to transition from "ascending" to "descending" faster than waiting for GPS altitude to confirm. It communicates over SPI at up to 24MHz — very fast because you might want IMU data at 1000Hz for high-rate logging.

BMP390 A MEMS barometric pressure sensor. Inside the chip is a tiny silicon membrane that flexes under atmospheric pressure — this flex is measured capacitively and converted to a pressure reading. Your firmware converts pressure to altitude using the barometric formula. At sea level pressure is ~1013 hPa. At 30km it's ~12 hPa. The BMP390 is accurate to about ± 0.5 hPa which corresponds to roughly ± 4 m altitude resolution at sea level, degrading at altitude. It also has a temperature sensor inside for pressure compensation. Important note: the BMP390 goes out of spec below ~300 hPa (~9km altitude) — below that pressure floor the reading clamps. So you use it for precise low-altitude readings (launch, landing, the first 9km) and rely on GPS altitude above that.

TMP117 A precision digital temperature sensor accurate to $\pm 0.1^{\circ}\text{C}$ across -40°C to $+150^{\circ}\text{C}$. Unlike the BMP390's internal temperature sensor which measures the chip's own die temperature, the TMP117 can be mounted externally — attached to a wire that pokes through the foam wall — to measure true ambient outside air temperature. At 30km altitude you'll see -60°C to -70°C . This is scientifically interesting data and also useful for monitoring battery temperature inside the box. Communicates over I2C.

TPS63070 A buck-boost DC-DC switching regulator. "Buck-boost" means it can both step voltage down (buck) and step it up (boost), maintaining a stable output regardless of whether input is above or below the output. Your 2S2P battery pack outputs 6.0–8.4V across its discharge curve. The TPS63070 takes that variable input and outputs a rock-solid 3.3V at up to 2A. If you used a simple linear regulator (LDO) instead, it would work when battery is above 3.3V but cut out as cells discharge — the buck-boost keeps running until the battery is nearly dead. Switching frequency is around 1–2MHz. Requires an external inductor (~2.2 μH), input cap, and output cap on the PCB.

microSD Card Holder A physical spring-loaded connector that accepts a microSD card. Connects to the STM32 over SPI — four pins: MOSI, MISO, SCLK, CS. Your firmware uses the FatFS library to write a FAT32 filesystem, logging sensor data, GPS coordinates, and flight events to a CSV or binary file every second. This is your black box — if radio fails, the SD card has everything. The SanDisk Industrial card is specifically rated for -40°C operation and write endurance much higher than consumer cards.

SWD Debug Header Serial Wire Debug — a 5-pin connector (VCC, GND, SWDIO, SWDCLK, NRST) that is the programming and debugging interface for the STM32. You connect an ST-Link programmer (costs ~\$10) to this header to flash firmware. Without it you cannot put code on the chip. During development you also use it to step through code line by line with a debugger. This stays on the PCB permanently but you only use it on the bench and on the pad before launch.

Raspberry Pi Zero 2W A complete Linux computer on a board the size of a stick of gum. Runs Raspberry Pi OS (Debian-based Linux). Has a camera interface (CSI-2), WiFi, and GPIO pins. Its only job in your system is: receive a trigger signal from the STM32 → capture a photo with the Pi Camera → compress it → run SSDV encoder → send packets back to STM32 → save full-res version to SD card. It takes ~30–45 seconds to boot, so you power it on the pad well before launch. Draws ~150–300mA at 5V while running.

Pi Camera Module 3 A camera module with a Sony IMX708 sensor (12MP), autofocus, and HDR capability. Connects to the Pi Zero via a CSI-2 ribbon cable — a high-speed differential serial interface that the Pi's Broadcom processor handles natively. libcamera (the Linux camera stack) drives it. You configure it to capture full-res JPEGs saved locally, plus a downscaled version for SSDV transmission. The wide angle lens (~66° FoV) is good for capturing Earth's curvature at altitude.

Runcam A self-contained action camera — has its own battery, own storage, own lens. You just mount it to the outside of the payload box, press record before launch, and retrieve the footage after recovery. Zero electrical integration with your system. It's there purely for wide-angle cinematic video footage that looks good in your demo video, separate from the engineering data the Pi Camera is capturing.

Communication Protocols — Every Device

SPI — ICM-42688-P, RFM95W, BMP390, microSD

These four share one SPI bus on the STM32. SPI is the right choice here for three distinct reasons depending on the device:

For the IMU (ICM-42688-P): you need speed. If you want IMU data at 1000Hz, each read is ~14 bytes (accel + gyro). At 1000Hz that's 14,000 bytes/second minimum. SPI at 8MHz handles this trivially. I2C at 400kHz fast mode would struggle.

For the LoRa module (RFM95W): the SX1276 inside only speaks SPI — it's not negotiable, it has no I2C interface. You write to its register map over SPI to configure spreading factor, bandwidth, transmit power, then hand it a packet buffer to transmit.

For the BMP390: it supports both SPI and I2C but SPI is preferred because you're already running a SPI bus and SPI is faster and simpler to share via chip selects.

For the SD card: SD protocol in SPI mode is the standard way to talk to SD cards from microcontrollers. It's the interface FatFS is written for. Fast enough for your logging rates.

Each SPI device gets its own dedicated CS (chip select) pin from the STM32. When you pull a device's CS pin low, it wakes up and listens. All others ignore the bus. So four SPI devices = four GPIO pins used as CS.

I2C — TMP117

I2C for the temp sensor makes sense for one reason: it only has two wires (SDA + SCL) and the TMP117 only supports I2C anyway. Since you're reading temperature at maybe 1Hz, the slow speed (400kHz) is completely irrelevant. If you had six temperature sensors you'd still use I2C and just give each a different address — that's I2C's strength, multiple devices on two wires.

UART — GPS (MAX-M10S), APRS (DRA818V)

GPS is UART because u-blox designed it that way — it streams continuous ASCII NMEA sentences whether you ask or not. UART is the natural fit for streaming data. You configure one of the STM32's USART peripherals to 9600 or 115200 baud, enable DMA (Direct Memory Access) so sentences are received into a buffer without CPU involvement, and parse them in firmware.

APRS (DRA818V) is UART for configuration — you send AT commands to set frequency, squelch, volume. Then separately you feed audio-frequency AFSK tones into its analog audio input pin, generated by the STM32's DAC or a

PWM output filtered to analog. So it's actually UART for control + DAC/PWM for the actual transmission data — a slightly unusual split.

GPIO — Pi Zero trigger, status LEDs

The STM32 pulls one GPIO pin HIGH to tell the Pi to take a photo. The Pi has a Python script watching that pin with a hardware interrupt — the moment it goes high, it captures. No protocol overhead, just a voltage level change. Simple and reliable.

The Pi also sends SSDV packets back to the STM32 — this goes over a UART link between the two boards, not GPIO. GPIO is just the trigger.

Board Architecture — How Many PCBs

This is a real design decision with tradeoffs. Here are the three realistic options:

Option A: Two PCBs (recommended)

PCB 1 — Flight computer. Everything except the Pi and camera. STM32, all sensors, both radios, power management, SD card, GPS module. Roughly 80×60mm, 4-layer.

PCB 2 — This is just the Pi Zero itself (which is a pre-made board). You're not designing PCB 2, you're buying it. The Pi sits on standoffs inside the box connected to your flight computer via a short JST cable carrying 5V power and a 3-pin UART/GPIO connector.

This is the right split. Your custom engineering is concentrated on PCB 1. The Pi is an off-the-shelf peripheral. Clean separation.

Final Recommendation: Two Boards, One Custom

Your custom 4-layer PCB (flight computer) contains: STM32, crystal, GPS module, RFM95W, DRA818V, ICM-42688-P, BMP390, TMP117, TPS63070, LDO 5V, microSD holder, SWD header, all passives. Target size: 70×60mm. This is the board you design in KiCad, send to JLCPCB, hand-solder, and bring up.

Pi Zero 2W (off the shelf) sits separately on standoffs or in a foam pocket, connected to your PCB by three wires: 5V, GND, and a UART TX/RX pair for SSDV packet handoff plus one GPIO trigger wire. That's it.

The Runcam is completely standalone — no electrical connection to anything.

This architecture means your entire custom design effort is focused on one board where you have full control, and the Pi handles camera complexity without requiring you to design around a Broadcom processor.

<https://octopart.com/part/stmicroelectronics/STM32F411CEU6>

BOM what we need to work on:

Battery: Fly on 4S2P Energizer Ultimate, but test on rechargeable 2S Li-ion battery

- For 2S Li-ion battery use Keppower 2S2P 18650 7000mAh 7.4V

For antennas:

GPS (active, 1575 MHz, SMA male):

- <https://www.amazon.com/MASWELL-Professional-Antenna-1575-42MHz-Semisp/dp/B08CCX3K6F>
- Alt (quality puck, narrowband L1 filter):
<https://www.proxicast.com/shopping/ant-190-010.html>

LoRa (915 MHz, SMA male whip):

- <https://www.amazon.com/Antenna-Connector-Meshtastic-Wireless-Transceiver/dp/B0GQH2G9NZ>
- Alt (flexible):
<https://www.amazon.com/915MHz-Antenna-10dBi-Range-Meshtastic/dp/B0DY7KVJ1S>

APRS (144 MHz HAB dipole, SMA):

- <https://shop.bigredbee.com/products/brb-vh-dipole-antenna>

Camera:

Links (ArduCAM 5MP Plus OV5642):

- Arducam direct:

<https://www.arducam.com/arducam-new-5mp-mini-spi-camera-module-for-arduino-uno-raspberry-pi-pico-and-other-mcu.html>

- SparkFun:

<https://www.sparkfun.com/arducam-5mp-plus-ov5642-mini-camera-module.html>

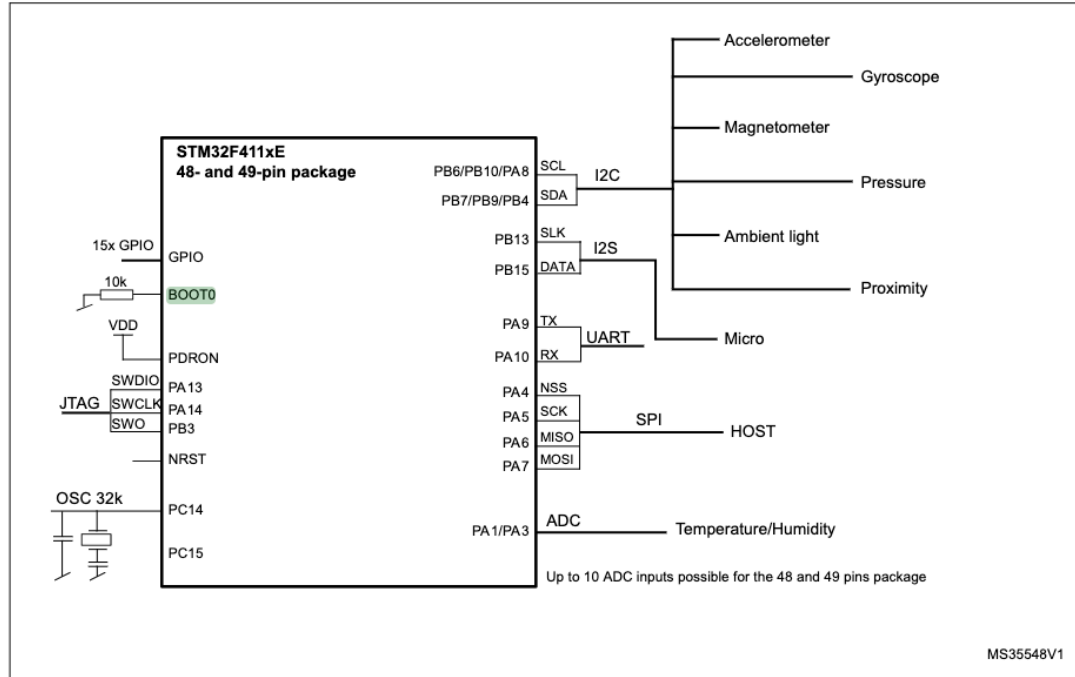
On/off switch:

TE Connectivity ALCOSWITCH TMS6T1B1M1QE

PCB Notes

B.2 Sensor Hub application example

Figure 60. Sensor Hub application example



[STM32F411CEU6 STMicroelectronics | Microcontrollers | DigiKey](#)
[ABM8W-12.0000MHZ-8-K1Z-T3 Abracon LLC | Crystals | DigiKey](#)

Parts Data

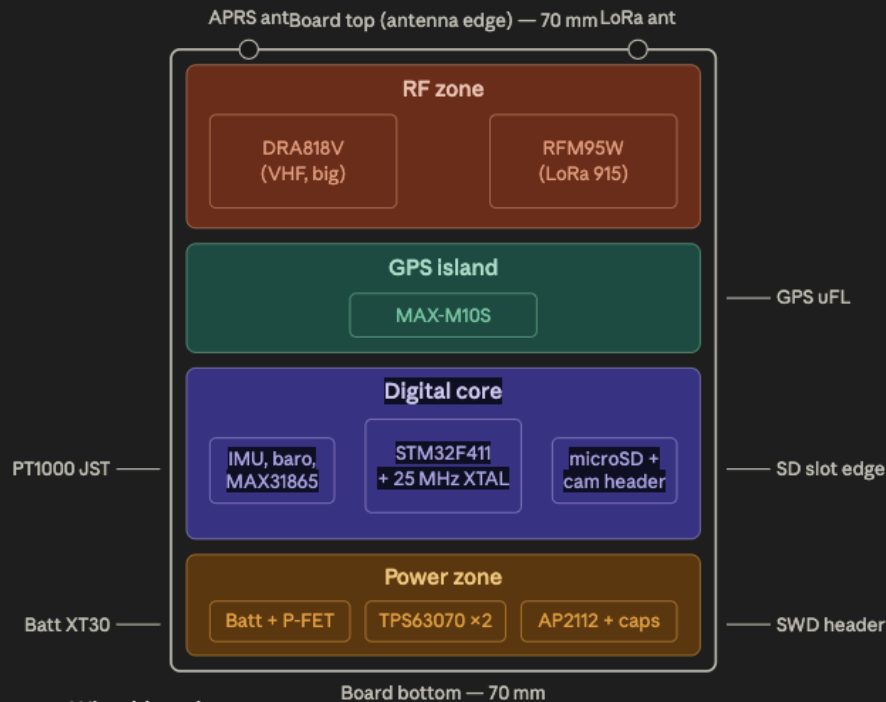
<i>Module/IC</i>	<i>Operating Temperatures</i>
STM32F411CEUx	Ambient temperatures: - 40 to +85 °C / - 40 to + 105 °C/ - 40 to + 125 °C Junction temperature: – 40 to + 130 °C
TPS63070RNMR	Operating Junction Temperature Range: -40 to +125 °C
AP2112K-3.3TRG1	Operating Junction Temperature Range: +150 °C Storage Temperature Range: -65 to +150 °C Lead Temperature: +260 °C (for soldering) Ambient Operation Temperature Range: -40 to +85 °C
SW_SPDT	no datasheet
ABM8W-12.0000MHZ-8-K1Z-T3	Operating Temperature Range: -40 to +125 °C Storage Temperature: -55 to +125 °C
Conn_01x05	no datasheet
ArduCAM_Mini_2MP_Camera_Shield_DS	-10 to +55 °C
NRST	no datasheet
BOOT0	no datasheet
MAX-M10S-00B	Ambient AND Storage AND Operating Temperature: -40 to +85 °C
RFM95W-915S2	Operational temperature range: -20 to +70 °C Absolute Maxes: -55 to +115 °C, Junction: +125 °C
DRA818V	Operating Temperature Range: -20 to +70 °C Storage Temperature: -40 to +90 °C
ICM-42688-P	Operating Temperature Range: -40 to +85 °C Storage Temperature Range: -40 to +125 °C
MS5611-01BA03	Operating Temperature: -40 to +85 °C Storage Temperature: -40 to +125 °C Maximum Soldering Temperature: 250 °C
MAX31865ATP_T	Operating Temperature Range: -40 to +125 °C Junction Temperature: +150 °C Storage Temperature Range: -65 to +150 °C Soldering Temperature (Reflow): +260 °C Lead Temperature (soldering, 10s): +300 °C

PT1000 Sensor	no datasheet
1040310811	no datasheet
micro_SD_detect	no datasheet

Layout strategy

Floorplan for your board

The key idea: **zone by noise behavior, not by schematic sheet**. Noisy things (RF transmitters, switching regulators) go on one side, sensitive things (GPS receiver, IMU) on the other, digital hub in the middle.



Why this order

RF at top: shortest trace to antennas, isolates transmit-side noise from receive-side GPS.

Power at bottom: switching noise pulled far from GPS and IMU. Star-ground to layer 2.

STM32 in the middle so no SPI/UART trace has to go all the way to the board — reach all sensors and radios.

Placement order

Do them in this exact order — later components fill around the anchors set by earlier ones:

1. Antenna connectors first. Place the LoRa uFL and APRS SMA (or uFL) on the top edge. Keep them $\geq 15\text{mm}$ apart — 915 MHz and 144 MHz are far apart in frequency but their harmonics can still desense each other. Snap to a nice grid so mechanical layout of your enclosure is easy.

2. The two radios, right next to their antennas. RFM95W's RF pin should be within $\sim 10\text{mm}$ of the uFL — the 50Ω trace between them is the single most sensitive trace on the board. DRA818V is a module with its own antenna pin; short and direct.

3. GPS module. Place under whatever will be its antenna path. If you're using an external active antenna via uFL, put the uFL on an edge with clear space around it. Keep the MAX-M10S itself at least $\sim 10\text{mm}$ from any switching regulator inductor.

4. STM32 in the center. This is your hub. Center it so no SPI CS line has to snake across the board. The 25 MHz crystal goes within ~5mm of the STM32 crystal pins, with a ground pour under it and no digital traces routed underneath.

5. Sensor cluster around STM32. IMU, MS5611, MAX31865 all share the SPI bus, so keep them within ~15mm of the STM32 to keep SPI traces short. MS5611 needs a small vent hole in your enclosure over it — mark that spot in your notes. The MS5611 also should not sit right on top of anything that generates heat (LDO, radios during TX).

6. microSD on a board edge so you can push a card in without disassembly. SD is on the shared SPI bus with 10kΩ CS pullup you flagged as critical.

7. Switching regulators, grouped and isolated. Both TPS63070s go in one corner of the bottom zone. Their high-di/dt loop (input cap → IC → inductor → output cap) must be as small as physically possible — this is the #1 EMI rule for buck-boost. AP2112K downstream, close to whatever it feeds (probably STM32 VDD).

8. Battery input + protection chain at the opposite side of the power zone from the switchers, so the input current path doesn't loop across the whole board.

9. Debug/SWD header on an edge, doesn't matter which — you just need to reach it with a programmer.

Key layout rules for this board

RF trace impedance. For the RFM95W to uFL trace, use a 50Ω coplanar waveguide with ground on both sides on layer 1 and full ground under it on layer 2. JLCPCB's impedance calculator (or KiCad's built-in [Board Setup → Impedance](#)) will give you the width for 1.6mm 4-layer stack — typically ~0.35mm trace with 0.2mm gap to side ground. Don't wing this; a 25Ω trace will kill your link budget by 6dB.

Ground plane is continuous. Layer 2 is a solid ground pour with no splits. When you route on layer 1 or 4, do NOT let a trace's return path have to jump over another trace on the plane layer. This is where 4-layer earns its keep.

Star ground for switchers. The TPS63070 GND pin, its input cap GND, and its output cap GND should meet at a single point that then vias down to layer 2 ground. Don't just spread them out and let ground flow through the plane.

Stitching vias. Along the perimeter of the board and around RF sections, drop GND vias every ~5mm connecting all four layers' grounds. This suppresses ground resonance and helps EMC.

Barometer keepout. No components within ~3mm of the MS5611 in x-y, and no ground pour directly around its pressure port. Airflow needs to reach the sensor's cavity.

Start with just steps 1-5 (antennas, radios, GPS, STM32, sensors). Show me that placement and I'll help you validate before you move on to power and routing.

Embedded Coding

Notes

System Core Setup

- SYS → Debug: Serial Wire
 - SWD (serial wire debug): hardware protocol for processors using two pins
 - enables SWD pins (PA13/PA14)
 - SWDIO (serial wire debug input/output)
 - bidirectional data pipeline
 - uses C code to spit values back and forth
 - SWCLK (serial wire clock)
 - synchronization wire
 - dictates when to read or write data on SWDIO
- Note: Might need an ST-Link Programmer for testing (what the debug is for)
 - might be able to use Nucleo-Board as stand-in according to Gemini

The Good News: You Already Own One

As we touched on earlier, while you *need* the hardware to program the board, you do not need to buy a new standalone programmer.

Because you have the STM32 Nucleo-F411RE listed in your BOM for the ground station, you already have an ST-Link hiding in plain sight.

The top quarter of every Nucleo board (the part with the USB plug) is a dedicated ST-Link programmer that can be used to program external custom boards:

1. Remove the two small CN2 jumpers on the Nucleo board. This severs the connection to the Nucleo's own MCU.
2. Run jumper wires from the SWD pins on the top half of the Nucleo directly to the 5-pin SWD header you are designing onto your custom KiCad board.
3. Hit "Run" in STM32CubeIDE. Your laptop will push the code through the Nucleo and directly into your custom flight computer!

- RCC (Reset and Clock Control) → HSE (High Speed Clock): Crystal/Ceramic Resonator
 - manages clock signals for main CPU and every communication protocol
 - HSE vs HSI: MCU already has internal clock (HSI; RC oscillator; 16 MHz) but is highly resistant to temperature changes; reason why we use HSE
 - HSE: usually piece of quartz of certain microscopic thickness that vibrates when voltage is applied to it; usually 8 MHz; temperature resistant
 - setting change dictates MCU PH0 and PH1 pins to use HSE over HSI
- Clock Configuration: Basically sends frequency through a phase-locked loop that divides and multiplies the frequency to get ideal values/outputs

Pin Configuration Notes

- Alternate Function: outside of CPU's manual control; so it can do better things usually
 - usually CPU gives instructions to middlemen (i.e. SPI hardware) located in MCU and sensor (peripherals)
- Chip Select
 - CS pins set as GPIO output as the MCU is outputting a constant voltage
 - CS pins use active-low logic (low voltage means activate) so set default output level to high
 - set maximum output speed to high because SPI is very fast
 - GPIO mode: Push-Pull instead of Open Drain (only one wire on the line)
 - Push-Pull has two switches connecting to 3V3 source and ground alternating between the two for desired effect
 - Open Drain has a resistor connected to 3V3 source and switch to ground but is slower because of resistor but can hold multiple open drain systems
 - GPIO is no pull up/no pull down (no need to tether it to a set voltage)
 - pull down connects wire to ground with internal spring/resistor
 - pull up connects wire to rail with internal spring/resistor
- IMU_INT1 (Interrupt 1)
 - sends voltage when ready to send new data
 - Pin Mode: GPIO_EXTI15 (external interrupt pin 15)
 - forcibly pauses code (while loop) and runs IMU code
 - went to NVIC (nested vector interrupt controller) and enabled EXTI interrupts
- USART TX/RX
 - GPIO Mode: Alternate Function Push/Pull (alternate to MCU C code dictating action)
 - start logic: active-low; data logic: active-high; uses start bit (low voltage) to start communication
 - set speed to low because UART is super slow and we shouldn't overshoot because it causes electric ringing, signal crosstalk, radio noise
 - use pull-up internal resistor to maintain 3V3
- APRS_AUDIO (PWM)
 - uses pulse width modification (have ding explain later)
 - something about RC low-pass filter
 - Timer 2 divides the clock signal from 100 MHz to 1200-2200 Hz (human noise)
 - also uses alternate function (not part of C main() code functions)
 - routed channel 1 to PWM Generation CH1 and clock source to Internal Clock
- SPI (CLK, MOSI, MISO pins)
 - Mode: Full-Duplex Master (Bidirection data flow)
 - Speed: High (SPI very fast)

- Alternate function Push Pull (MISO labeled this way too lazily but its overridden)
- LoRa RESET/DIO0 (Digital I/O 0)
 - reset just resets the LoRa chip if it freezes due to temperature everything normal here
 - for DIO use External Interrupt because it's LoRa's status pin on data transmission
- APRS PTT/PD
 - ask ding about this
 - Low speed, Low output level
- microSD_detect
 - checks if SD card is empty (3V3) or inserted (0V) because of insertion spring
 - switch however is open piece of metal, making it float requiring pull up resistor
- I2C (SDA/SCL)
 - serial data and serial clock pins need open-drain because protocol overlaps wires
 - active low passive high (SDA)

All things to test in code:

- GPS
- IMU
- Temperature Sensor
- Barometer (?)
- LoRa
- APRS

Livestream Proposal

Proposal

Instead of taking pictures of the curvature of the Earth, by using solely the MCU paired with the camera module, with or without the use of the Raspberry Pi module to process imaging, we move towards livestreaming the entire process using a WiFi card and having the Raspberry Pi module run Open HD.

Abstract Explanation

Standard Wifi (TCP): Uses transmission control protocol to perfectly send 1s and 0s in order

- Process: Sends packet → Waits for acknowledgement → Sends another packet
- If corruption occurs, process rewinds and resends packet
- At high altitudes, constant corruption is expected making TCP unreliable (will cause long waits for acknowledgement and many resends, eventually causing video to freeze)

Open HD (UDP): Acts like a user datagram protocol using packet injection instead

- Process: Sends packets without waiting for acknowledgement
- HD only changes the pixels in the frame that changed, so corruption would just assign the wrong pixels (causing blurry or laggy stream basically)
- Not the standard because it jams other networks (no waiting for other networks) and no address (raw radio frequency)
- Consequences: Frames are sometimes sent out of order (raspberry pi deletes them), messed up pixels (blurry stream), frames might be not sent (gone forever)

Raspberry Pi:

- Would in theory exist on a separate board from Custom Kicad PCB
- Gemini's Recommendation: 40-pin GPIO Ribbon Cable
 - very good for thermal control (boards far from each other)
 - good for RF jamming avoidance (antennas distanced from each other)
 - can help with inertia of entire balloon and stabilizing motion
- Add 40-pin female GPIO header to custom board
- Plugging in: Camera to CSI port; USB Open HD antenna to blue USB 3.0; custom board on top
- Flow of Data: STM32 takes GPS and altitude data and sends it to **MCU_TO_PI_TX** through physical header connector, Pi takes live video from camera, overlays MCU data like HUD, compresses it, blasts it out of USB WiFi card using Open HD packet injection

Engineering Explanation

Phase 1 (Ingestion):

- Light hits the HD camera, creating massive, raw stream of pixel data
- Camera is plugged into CSI (camera serial interface) via ribbon cable
 - wires camera directly to RSP's GPU, bypassing CPU entirely
 - allows RSP to ingest raw video at gigabits per second zero latency

Phase 2 (Compression):

- Done via RSP's GPU crushing raw video using H.264/H.265 encoding
 - looks at video frame, sees pixels that haven't changed since last frame and deletes them from data stream
 - RSP's CPU takes UART telemetry data from STM32 (altitude, gps) and chemically burns numbers into video frames as on-screen display
 - results in highly compressed data

Phase 3 (Transmission)

- using OpenHD we send the compressed data from the RSP to the WiFi card which transmits the data using radio waves using its antenna
 - OpenHD is a custom Linux operating system that intentionally disables the WiFi card's network regulators and does not provide the data with IP address
 - OpenHD gives the data a "radiotap" header before sending off to WiFi card and hacks the WiFi card to blindly throw radiotap data into its RF amplifier
 - without OpenHD the WiFi card would require sending the data in a regulated way (obeying federal network regulations, requiring IP addresses for sending data)

Questions

How do we want the camera to be angled?

How do we want to connect the Raspberry Pi? (HAT, Wire Harnessing, Ribbon Cable)

Can we handle the expanded power usage? (estimated from 2 watts to 9-13 watts)

Battery Proposal

Proposal

Recommendation A: Molicel 18650s (Rechargeable)

- High drain cells with good internal chemistry
- Maintenance down to -20 degrees Celsius
- Capacity: Pack of 2S2P of these provides ~5600 mAh
 - will likely need more if using OpenHD
 - highly viable due to not using RSP and Open HD

Recommendation B: Energizer Ultimate Lithium AA (Aerospace Standard)

- Maintenance down to -40 degrees Celsius
- Very light (weight wise)
- Will have to use 2S4P sled

DELETE TAB WHEN DECISION MADE

specifications for BOM

APRS:

Resistors (all 0603):

- **Material:** thick-film (standard, cheap — what JLCPCB stocks by default)
- **Tolerance:** 1% (except R20 divider — see below)
- **Power:** 0.1W (0603 default) is plenty; these carry μA –mA
- R11, R12 (PTT/PD pull-ups) 10k · R18 1k · R19 10k · R20 (divider shunt — your tuning R, currently 470) · R30 1k
- **R19/R20 specifically:** use **1% or better** since they set the audio level/deviation ratio

Caps (ceramic MLCC, 0603 except bulk):

- **Dielectric:** **C0G/NP0** for the audio-path caps (C35 10nF, C36 100nF coupling, C47) — C0G is stable, low-distortion, matters for clean audio. **X7R** is acceptable if C0G isn't available in that value/size.
- **Decoupling caps** (C37 100nF at VBAT): **X7R** fine
- **Bulk cap** (C23 at VBAT, 10 μF): **X7R or X5R, 0805**, voltage $\geq 10\text{V}$ (rail is 4.1V, want margin for the burst)
- **Voltage rating:** $\geq 16\text{V}$ on the small ones (cheap, gives DC-bias margin)

On JLCPCB, search/filter for:

- Resistors: **0603, thick film, 1%, "Basic Part"** (Basic parts = no extra assembly fee)
- Audio caps: **0603, C0G/NP0, 50V** — but C0G in 100nF may only be "Extended" (costs more); if so, drop to **X7R Basic**
- Decoupling/bulk: **0603 X7R 50V** and **0805 X7R 10 μF 25V**, prefer **Basic Parts**